

SOFTWARE MANUAL

Dual-LDA Workflow for Harnessing Cross-Sensitivity

Module-by-module documentation of the Jupyter notebook implementation

Parameter-agnostic terminology

Parameter A = first physical variable

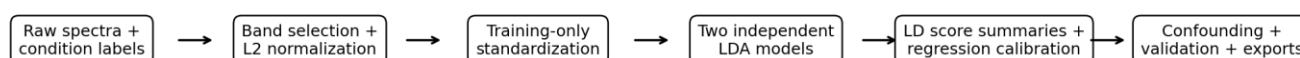
Parameter B = second physical variable

Prepared from the actual notebook source code with the help of ChatGPT

1. Introduction

This manual documents a supervised multivariate analysis workflow designed to recover two independently varying physical parameters from cross-sensitive spectral measurements. The implementation uses two separate Linear Discriminant Analysis (LDA) models: one trained to discriminate classes of Parameter A and one trained to discriminate classes of Parameter B. The first discriminant score from each model is then summarized at the experimental-condition level and calibrated against the corresponding continuous ground-truth parameter.

The notebook is not a single end-to-end software package with encapsulated modules; rather, it is a sequential analysis notebook whose headings define functional modules. Several export blocks and alternative preprocessing blocks are stored as Markdown cells containing executable Python. This manual describes the intended function of each section, the data dependencies between sections, and the implementation-specific assumptions that must remain consistent for correct execution.



Stage	Primary operation	Key result
1.0	Load spectral matrices and condition-level ground truth	Training and validation data objects
1.1	Select spectral band(s), L2-normalize each spectrum, transpose	Sample × feature matrices
1.2	Bin continuous labels and expand labels per spectrum	Per-spectrum class labels
2.0	Fit training scaler and two LDA models	Two latent discriminant spaces
2.1–2.3	Inspect variance, loadings, score separation and condition statistics	Model diagnostics and LD1 means/stds
2.4–2.6	Fit calibrations and assess cross-sensitivity	Parameter response functions and confounding metrics
3.0–3.2	Validate predictions and audit leakage	Independent prediction statistics
4.0	Export selected processed spectra	Publication/external-analysis files

2. Terminology and variable mapping

To make the manual reusable, the two experimentally controlled quantities are referred to as Parameter A and Parameter B. The notebook itself uses temperature and stirring-rate/RPM variable names, and some legacy export blocks use pressure-related names. The table below provides the mapping required to interpret the code.

Generic term	Principal notebook variables	Meaning
Parameter A	temperature_training_labels, temperature_validation_labels, LDA_T, LD_T_projection, LD1_T_means	First controlled physical parameter and its dedicated LDA model
Parameter B	rpm_training_labels, rpm_validation_labels, LDA_RPM, LD_RPM_projection, LD1_RPM_means	Second controlled physical parameter and its dedicated LDA model
Training spectra	raw_training_spectra_df →	Spectra used to fit the scaler, LDA

	transposed_training_spectra → scaled_training_spectra	models and regression calibration
Validation spectra	raw_validation_spectra_df → transposed_validation_spectra	Independent spectra transformed using training-derived objects
Condition	One unique pair of Parameter A and Parameter B values	Experimental setting represented by multiple spectra in training
LD1-A / LD1-B	First column of each LDA projection	Primary scalar latent response used for calibration

3. Data architecture and assumptions

The imported spectral CSV files are assumed to use rows as wavelength or feature positions and columns as individual spectra. Label CSV files are assumed to use one row per experimental condition. The notebook therefore transposes normalized spectral matrices before scikit-learn processing, because scikit-learn expects rows to represent samples and columns to represent features.

Object	Expected orientation	Use
raw_*_spectra_df	features × spectra	Raw imported intensity matrix
norm_*_spectra	features × spectra	Per-spectrum L2-normalized matrix
transposed_*_spectra	spectra × features	Input to StandardScaler and LDA
scaled_training_spectra	spectra × standardized features	Training matrix for both LDA models
expanded_*_training_labels	one class label per spectrum	Supervised LDA targets
LD_*_projection	spectra × discriminant components	Latent-score matrices
LD1_*_means / stds	one mean and standard deviation per condition	Condition-level calibration responses

Critical ordering assumption: spectra belonging to the same condition must be contiguous, and conditions must appear in exactly the same order as the rows of the training label file. Module 2.3 computes condition statistics by positional slicing rather than by an explicit condition identifier.

4. Module 1.0 — Load data and ground-truth labels

Notebook reference: Cell 1

Purpose

Load the training and validation spectral matrices and extract the two continuous ground-truth parameter vectors used throughout the workflow.

Inputs

- Training spectral CSV: each column is one spectrum.
- Validation spectral CSV in the same feature order as training.
- Training ground-truth CSV: one row per experimental condition.
- Validation ground-truth CSV: one row per validation condition.

Processing

1. Read all four CSV files with `pandas.read_csv(..., header=None)`.
2. Store spectra as `raw_training_spectra_df` and `raw_validation_spectra_df`.
3. Extract Parameter A from column 1 and Parameter B from column 0 of each label table.
4. Retain the labels as NumPy arrays for binning, plotting, regression and validation.

Outputs

- Four pandas DataFrames containing raw spectra and labels.
- `temperature_*_labels` and `rpm_*_labels` arrays, interpreted generically as Parameter A and Parameter B.

Role in the workflow

This module establishes all raw inputs. Every later module depends either directly or indirectly on these objects.

Implementation notes

- File paths are hard-coded absolute Windows paths and must be updated when the notebook is moved.
- The code does not validate missing values, feature count agreement, condition count, or label order. These checks should be added before routine reuse.

5. Module 1.1 — Spectral preprocessing

Notebook reference: Cells 3, 5 and 7

Purpose

Select the informative spectral region, normalize every spectrum to unit Euclidean norm, and transpose the matrix into the sample-by-feature orientation required by scikit-learn.

Inputs

- Raw training and validation spectral DataFrames.
- One of three alternative band-selection configurations: stitched green+red bands, green only, or red only.

Processing

5. Select feature rows using fixed positional slices. The stitched option concatenates two separated spectral windows; the single-band options retain only one window.
6. Reset row indices after concatenation in the stitched-band version.
7. Normalize each spectrum independently using $x/\|x\|_2$, removing absolute-intensity scale while preserving relative spectral shape.
8. Transpose normalized matrices so rows represent spectra and columns represent wavelength/features.
9. Plot representative normalized spectra as a visual sanity check.

Outputs

- `norm_training_spectra` and `norm_validation_spectra`.
- `transposed_training_spectra` and `transposed_validation_spectra`.
- Diagnostic plots of normalized spectra.

Role in the workflow

This stage defines the information presented to both LDA models. Band selection and normalization are therefore part of the model specification and must be identical for training and validation.

Implementation notes

- Cells 3 and 5 are Markdown cells containing Python; they must be converted to Code cells to execute.
- The three alternatives overwrite the same variable names. Only one preprocessing branch should be executed for a given model.

- The plotting loop uses `norm_training_spectra[i]`, which indexes rows/features in a DataFrame rather than `spectra/columns`; plotting `transposed_training_spectra.iloc[i, :]` is clearer.
- L2 normalization will divide by zero if an all-zero spectrum is present; a zero-norm guard is advisable.

6. Module 1.2 — Manual binning of ground truth

Notebook reference: Cell 9; class-balance export in Cell 11

Purpose

Convert continuous condition-level parameter values into discrete class labels required for supervised LDA, then replicate each condition label to match the number of spectra collected under that condition.

Inputs

- Continuous Parameter A and Parameter B arrays from Module 1.0.
- Manually defined bin edges for each parameter.
- `spectra_per_condition_training`, set to 106 in the notebook.

Processing

10. Use `numpy.digitize` to assign each condition to a Parameter A bin and a Parameter B bin.
11. Repeat each condition-level bin identifier 106 times to create one class label per training spectrum.
12. Plot histograms of the expanded class labels to inspect class balance.
13. Print a condition-to-bin mapping for manual verification.
14. Optionally export bin counts as a CSV table.

Outputs

- `temperature_bins` and `rpm_bins` at condition level.
- `expanded_temperature_training_labels` and `expanded_rpm_training_labels` at spectrum level.
- Class-balance histograms and optional CSV summary.

Role in the workflow

The two expanded label vectors are the targets used to train the independent LDA models. The binning scheme controls the supervised class structure and therefore strongly affects the discriminant directions.

Implementation notes

- The number of repeated labels must exactly equal the number of training spectra.
- `numpy.digitize` may create class 0 or a class above the last intended class when values lie outside the edge range.
- The export block refers to `pressure_bins`, which is not defined in the principal workflow; it should use `rpm_bins` or a consistently renamed Parameter B object.

7. Module 2.0 — Compute all LDA components

Notebook reference: Cell 13

Purpose

Fit a feature standardization transform on the training spectra, train two independent LDA models, and project the training data into each model's discriminant space.

Inputs

- `transposed_training_spectra`.
- Expanded per-spectrum classes for Parameter A and Parameter B.

Processing

15. Fit `StandardScaler` on training spectra only, centering each feature and scaling it to unit variance.
16. Fit `LDA_T` to discriminate Parameter A classes.
17. Fit `LDA_RPM` to discriminate Parameter B classes using the same standardized spectra.
18. Transform the training matrix with each fitted LDA model to obtain all available discriminant components.

Outputs

- scaler fitted on training data.
- `LDA_T` and `LDA_RPM` fitted estimators.
- `LD_T_projection` and `LD_RPM_projection` score matrices.

Role in the workflow

This is the central modelling stage. It creates two different supervised coordinate systems from the same spectra, each optimized for separation along one physical parameter.

Implementation notes

- The maximum number of LDA components is $\min(\text{number of features}, \text{number of classes} - 1)$.
- `StandardScaler` must never be re-fit on validation data.
- The LDA models are classifiers used here primarily as supervised feature extractors.

8. Module 2.1 — Scree plots, explained variance and loading vectors

Notebook reference: Cells 15 and 17

Purpose

Characterize how discriminant information is distributed across latent components and identify which spectral features contribute to the first and second discriminant axes.

Inputs

- A fitted LDA model with `explained_variance_ratio_` and `scalings_` attributes.

Processing

19. Read the explained variance ratio for every LD component.
20. Plot a scree curve against component number.
21. Display a table of component-wise explained variance.
22. Extract the first loading/weight vector from `scalings[:,0]` and plot it against feature index.
23. If available, extract and plot the second loading vector.
24. Export variance ratios and loading vectors to CSV for both models.

Outputs

- Scree plots.
- Explained-variance tables.
- LD1 and optional LD2 loading plots.

- CSV files containing variance ratios and weight vectors.

Role in the workflow

The module supports component selection and mechanistic interpretation. Dominant loading regions indicate which processed spectral features drive class separation.

Implementation notes

- The exported “weights” are LDA scaling coefficients in standardized feature space; interpretation should account for preprocessing and feature scaling.
- The export block uses LDA_P, whereas the main model is named LDA_RPM. Consistent naming is required.

9. Module 2.2 — LDA quality checks and score-space diagnostics

Notebook reference: Cells 19, 21 and 22

Purpose

Visualize separation of manually defined classes in the first two discriminant dimensions and export the score coordinates for external plotting or analysis.

Inputs

- LD_T_projection and LD_RPM_projection.
- Expanded class labels.

Processing

25. Check that at least two LD components are available.
26. Create LD1-versus-LD2 scatterplots coloured by the relevant class label.
27. Export a tidy/non-segregated table containing LD1, LD2 and class bin.
28. Optionally pivot scores into a wide table with paired LD1/LD2 columns for each bin.

Outputs

- Class-coloured score plots.
- Long-format and wide-format CSV files of LD1/LD2 scores.

Role in the workflow

This module provides a qualitative assessment of within-class compactness, between-class separation and overlap. The exports support publication graphics in external software.

Implementation notes

- Despite importing classification_report, StratifiedKFold, cross_val_score and ConfusionMatrixDisplay, the notebook does not actually calculate classification accuracy, cross-validation scores or a confusion matrix.
- Several export cells use y_train_T, y_train_P and LD_P_projection, which are not defined in the principal naming scheme. They should map to expanded_temperature_training_labels, expanded_rpm_training_labels and LD_RPM_projection.

10. Module 2.3 — Condition-level LD1 mean and standard deviation

Notebook reference: Cell 24

Purpose

Collapse spectrum-level LD1 scores into one mean response and one dispersion estimate for every experimental condition.

Inputs

- LD_T_projection and LD_RPM_projection.
- Condition-level continuous labels.
- spectra_per_condition_training.

Processing

29. For condition i , slice the contiguous score range $i \times N$ to $(i+1) \times N$.
30. Extract the first discriminant score from each model.
31. Calculate the arithmetic mean and population standard deviation for each condition.
32. Plot both condition-level response surfaces as 3D scatterplots against Parameter A and Parameter B, with vertical ± 1 standard-deviation bars.

Outputs

- LD1_T_means, LD1_T_stds, LD1_RPM_means and LD1_RPM_stds.
- Two diagnostic 3D scatterplots.

Role in the workflow

The condition means become calibration variables in Modules 2.4, 2.5 and 3.0. The standard deviations quantify repeatability within each experimental condition.

Implementation notes

- The calculation assumes fixed and equal spectra per condition and contiguous ordering.
- np.std uses the population definition (ddof=0).
- Explicit condition identifiers would make the grouping more robust than positional slicing.

11. Module 2.4 — Calibration surface fitting

Notebook reference: Cells 26 and 28

Purpose

Fit empirical relationships between each condition-level LD1 response and its corresponding continuous physical parameter, evaluate the fitted response across a two-parameter grid, and export the original scatter data.

Inputs

- Continuous Parameter A and Parameter B condition labels.
- LD1 means and standard deviations from Module 2.3.

Processing

33. Transform labels to delta coordinates using fixed reference offsets.
34. Fit LD1-A with a univariate linear regression in Parameter A.
35. Fit LD1-B with a third-degree polynomial in Parameter B.
36. Calculate fitted LD1 values and coefficient of determination (R^2).
37. Generate a meshgrid spanning specified parameter ranges.

38. Evaluate each univariate calibration over the meshgrid, producing a visually planar/extruded surface independent of the other parameter.
39. Plot condition data and fitted surfaces in 3D.
40. Export condition labels, LD1 means and LD1 standard deviations as CSV files.

Outputs

- T_model and RPM_model calibration objects/coefficients.
- R^2 statistics.
- LD1 response surfaces and long-format surface tables in memory.
- LD1-A and LD1-B scatter CSV files.

Role in the workflow

This module provides the forward calibration relationship: physical parameter → expected LD1 response. Module 3.0 later fits the inverse relationship LD1 → physical parameter for prediction.

Implementation notes

- The comments/headings in the notebook are inconsistent about whether the Parameter A relation is linear, quadratic or cubic. The executed code in Cell 26 is linear for Parameter A and cubic for Parameter B.
- The surfaces do not include a cross-term; each response is deliberately independent of the non-target parameter.
- The grid limits and delta offsets are hard-coded and must match the experimental design.

12. Module 2.5 — Regression-model justification

Notebook reference: Cell 30

Purpose

Compare polynomial degrees 1, 2 and 3 for each LD1-to-parameter relationship using fit quality and residual behaviour.

Inputs

- Condition-level LD1 means.
- Corresponding continuous ground-truth labels.

Processing

41. Generate polynomial design matrices of degrees 1, 2 and 3.
42. Fit ordinary least-squares regression for each degree.
43. Calculate MSE, RMSE, R^2 and an AIC-like criterion $n \cdot \ln(\text{MSE}) + 2k$.
44. Plot the observed data with a smooth fitted curve.
45. Calculate residuals as predicted LD1 minus observed LD1.
46. Plot residuals with zero and $\pm 3\sigma$ reference lines.

Outputs

- Six diagnostic panels per parameter: three fits and three residual plots.
- Quantitative comparison statistics embedded in plot titles.

Role in the workflow

This stage provides evidence for selecting the simplest regression degree that captures the systematic response without unnecessary complexity.

Implementation notes

- The AIC expression is a simplified Gaussian-error comparison and omits constants that cancel only when comparing models on the same data.
- No held-out or cross-validated model selection is performed here; the comparison is in-sample.
- The module title does not match the code comments elsewhere; model choice should be documented from the executed degree settings.

13. Module 2.6a — Visual confounding analysis

Notebook reference: Cell 32

Purpose

Test whether the latent response intended for one parameter changes systematically with the other parameter.

Inputs

- Condition-level LD1 means.
- Continuous parameter labels and binned class assignments.

Processing

47. Plot LD1-A mean against Parameter B, colouring points by Parameter A bin.
48. Plot LD1-B mean against Parameter A, colouring points by Parameter B bin.
49. Inspect within-bin trends along the non-target parameter axis.

Outputs

- Two confounding-diagnostic scatterplots.

Role in the workflow

An ideal decoupled response shows little or no dependence on the non-target parameter after accounting for the target parameter class. Visible slopes or structured shifts indicate residual cross-sensitivity.

Implementation notes

- This is primarily a visual diagnostic; no slope, partial correlation or statistical significance test is calculated.
- Colouring by target bins helps separate genuine target response from non-target drift.

14. Module 2.6b — Loading-vector angle and score correlation

Notebook reference: Cells 34, 35 and 37

Purpose

Quantify similarity between the two LD1 loading directions and statistical association between the two sets of LD1 scores.

Inputs

- First loading vector from each LDA model.

- First discriminant score for every training spectrum.

Processing

50. Compute cosine similarity between the LD1 loading vectors.
51. Convert cosine similarity to an angle in degrees.
52. Interpret values near zero cosine (approximately 90°) as near-orthogonal feature-space directions.
53. Compute Pearson correlation between spectrum-level LD1-A and LD1-B scores.
54. Plot one score against the other, coloured by Parameter A class.
55. Export long and bin-segregated score-pair tables for external plotting.

Outputs

- Cosine similarity and angle.
- Pearson correlation coefficient.
- LD1-A versus LD1-B scatterplot.
- Long-format and wide-format CSV exports.

Role in the workflow

The loading-vector comparison addresses whether the two models depend on similar spectral directions, whereas score correlation addresses whether their sample responses co-vary. Together they provide complementary measures of decoupling.

Implementation notes

- The sign of an LDA axis is arbitrary. Therefore cosine sign alone is not physically meaningful; $|\cosine|$ or the acute angle is more appropriate for similarity of directions.
- The notebook's note reports a strong non-orthogonality for one execution, indicating that the two latent directions share substantial spectral information.
- The export block again uses legacy names such as `ld1_P_scores` and `y_train_T`.

15. Module 3.0 — Independent validation and inverse regression

Notebook reference: Cells 39, 41 and 43

Purpose

Apply the training-derived scaler and LDA models to independent spectra, convert validation LD1 scores into physical-parameter predictions, and evaluate prediction accuracy and residuals.

Inputs

- Fitted training scaler.
- Fitted LDA models.
- Validation spectra preprocessed identically to training.
- Training condition-level LD1 means and continuous labels.
- Validation ground-truth parameter values.
- Chosen polynomial degree for each inverse calibration.

Processing

56. Transform validation spectra using `scaler.transform` without refitting.
57. Project validation spectra with the relevant LDA model and extract LD1.
58. Fit a polynomial regression from training LD1 means to the training continuous parameter.

59. Predict the parameter for each validation LD1 score.
60. Calculate RMSE and R^2 against validation ground truth.
61. Plot predicted versus true values with a 1:1 reference line.
62. Calculate and plot residuals as predicted minus true.
63. Optionally group validation spectra into fixed blocks, calculate LD1 means/stds, and export them.
64. Export true values, predictions and residuals to CSV.

Outputs

- `val_T_true/pred` and `val_RPM_true/pred` arrays.
- RMSE and R^2 for each parameter.
- True-versus-predicted and residual plots.
- Validation CSV files and optional block-level LD1 summary.

Role in the workflow

This module tests generalization to independent measurements and is the principal performance assessment of the predictive workflow.

Implementation notes

- In the executed validation code, Parameter A uses degree 1 and Parameter B uses degree 3.
- `spectra_per_condition` is set to 1 inside `validate_regression`; validation labels must therefore already have one entry per spectrum unless the function is revised.
- Cell 41 uses legacy pressure variable names that do not match Cell 39.
- Cell 43 assumes 350 spectra per validation block and seven blocks, which differs from the one-spectrum-per-condition assumption in `validate_regression`. These represent alternative validation layouts and should not be mixed without adjustment.

16. Module 3.1 — Violin plots of prediction distributions

Notebook reference: Cell 45

Purpose

Visualize the distribution, spread and potential bias of predicted values within each true validation condition.

Inputs

- True validation labels.
- Predicted validation values.

Processing

65. Construct one tidy DataFrame per parameter containing true and predicted values.
66. Use `seaborn.violinplot` to estimate and display the prediction distribution for each true condition.
67. Place the two parameter plots side by side for comparison.

Outputs

- Violin plots showing prediction distributions by true condition.

Role in the workflow

These plots complement RMSE and R^2 by showing heteroscedasticity, multimodality, skew, overlap and condition-specific bias.

Implementation notes

- The cell uses `T_validating_labels`, `P_validating_labels`, `val_P_pred` and pressure terminology that do not match the principal variable names.
- The repeat factor assumes a constant number of validation spectra per condition.

17. Module 3.2 — Data-leakage audit

Notebook reference: Cell 47

Purpose

Perform basic checks intended to confirm separation of training and validation data and correct reuse of training-derived preprocessing/model objects.

Inputs

- Training and validation spectral matrices.
- Fitted scaler and both fitted LDA models.
- Training LD1 means and ground-truth arrays.

Processing

68. Hash every spectrum with SHA-256 and compare training and validation hash sets for exact duplicates.
69. Call `scaler.transform` on validation data to confirm compatibility without fitting.
70. Check that both LDA objects expose `classes_`, indicating fitted estimators.
71. Compare the length of supplied LD1 means against both training label vectors.
72. Print pass/warning messages.

Outputs

- Console audit report.

Role in the workflow

The audit is a useful final safeguard against the most obvious forms of sample reuse and preprocessing misuse.

Implementation notes

- `hash_array` uses `arr.tobytes` implicitly only if the array buffer is accepted; robust code should use `hashlib.sha256(np.ascontiguousarray(arr).tobytes()).hexdigest()`.
- The function accepts only one `train_ld1_means` array but uses it to check both parameters; separate LD1-A and LD1-B arrays should be supplied.
- The fitted-state checks do not prove that validation data were excluded from model fitting.
- Near-duplicate spectra and grouped/replicate leakage are not detected by exact hashing.

18. Module 4.0 — Selected processed-spectrum visualization and export

Notebook reference: Cells 49, 51 and 53

Purpose

Inspect representative standardized spectra and export selected contiguous blocks of training or validation spectra in a transposed CSV format suitable for external plotting software.

Inputs

- scaled_training_spectra.
- Validation spectra transformed by the training scaler.
- Configured block size, block indices and output directory.

Processing

73. Plot selected standardized training spectra by row index.
74. Convert 1-based block identifiers to zero-based row slices.
75. Extract the requested spectrum rows from the scaled matrix.
76. Transpose each selection so spectra become columns and features become rows.
77. Write headerless CSV files whose filenames encode block and row ranges.
78. Skip blocks that fall outside the available spectrum count.

Outputs

- Diagnostic plot of selected standardized spectra.
- One CSV file per selected training or validation block.

Role in the workflow

This export stage supports publication plotting, comparison with external software and archival of the exact processed inputs used by the models.

Implementation notes

- The training export refers to training_spectra_df, while the imported object is raw_training_spectra_df.
- The validation export similarly refers to validation_spectra_df instead of raw_validation_spectra_df.
- The selected training row indices 0, 700 and 1400 assume a sufficiently large dataset.

19. Function reference

Function	Module	Function
temperature_bins_function(data)	1.2	Assign Parameter A values to manually defined bins using numpy.digitize.
rpm_bins_function(data)	1.2	Assign Parameter B values to manually defined bins.
plot_scree_and_weights(lda_model, title_prefix)	2.1	Plot explained variance and LD1/LD2 loading vectors.
export_lda_results(lda_model, prefix, export_dir)	2.1 export	Write explained variance and loading vectors to CSV.
export_ld_pairs_wide(proj, bins, prefix, outname)	2.2 export	Pivot LD1/LD2 scores into paired columns by class.
run_regressions(ld1_means, true_labels, label, unit)	2.5	Compare polynomial degrees 1–3 using fit and residual diagnostics.
validate_regression(...)	3.0	Fit inverse calibration on training LD1 means and predict validation values.
hash_array(arr)	3.2	Create a spectrum hash for exact-overlap detection.
check_data_leakage(...)	3.2	Run the notebook's basic leakage checks.
block_row_slice(block_k, block_size, total)	4.0	Convert a 1-based block number into a safe row interval.

20. Mathematical methodology

20.1 Per-spectrum L2 normalization

For a spectrum x with feature intensities x_j , the normalized spectrum is $\tilde{x} = x / \sqrt{\sum_j x_j^2}$. This removes overall magnitude and emphasizes relative spectral shape. It is applied independently to every training and validation spectrum.

20.2 Feature standardization

For each feature j , StandardScaler estimates the training mean μ_j and standard deviation σ_j and transforms x_j to $z_j = (x_j - \mu_j)/\sigma_j$. The same μ_j and σ_j are reused for validation spectra.

20.3 Linear Discriminant Analysis

LDA finds linear directions that maximize separation between predefined classes relative to variation within classes. In matrix terms, discriminant directions are associated with the generalized eigenproblem $S_B w = \lambda S_W w$, where S_B and S_W are the between-class and within-class scatter matrices. Two models are fitted independently because the class definitions for Parameter A and Parameter B differ.

20.4 Condition-level score statistics

For condition c with N spectra and LD1 scores s_i , the mean response is $\bar{s}_c = (1/N)\sum_i s_i$ and the notebook standard deviation is $\sqrt{[(1/N)\sum_i (s_i - \bar{s}_c)^2]}$. These statistics summarize sensitivity and repeatability at each condition.

20.5 Calibration and prediction

The notebook evaluates linear, quadratic and cubic polynomial relations. Module 2.4 fits forward functions from a physical parameter to LD1. Module 3.0 fits inverse functions from LD1 to a physical parameter for validation prediction. These are distinct regression directions and need not yield identical residual behaviour.

20.6 Performance metrics

R^2 measures the fraction of variance in the reference values explained by predictions. RMSE is the square root of the mean squared prediction error and remains in the physical units of the predicted parameter. Residuals are defined as predicted minus observed.

20.7 Cosine similarity and correlation

Cosine similarity compares the orientation of the two LD1 loading vectors: $\cos\theta = (w_A \cdot w_B) / (||w_A|| ||w_B||)$. Pearson correlation compares the two score vectors across samples. The former concerns feature-space directions; the latter concerns covariance of sample responses. Because discriminant-axis sign is arbitrary, directional similarity is often summarized by $|\cos\theta|$ or the acute angle.

21. Recommended execution order

Order	Section	Requirement before execution
1	1.0	Correct file paths and compatible CSV formats
2	Exactly one 1.1 branch	Select stitched, first-band-only or second-band-only preprocessing
3	1.2	Set bin edges and exact spectra-per-condition count
4	2.0	Verify expanded-label length equals

		training sample count
5	2.1–2.3	Use consistent LDA_RPM/LDA_T naming
6	2.4–2.6	Confirm intended polynomial degrees and physical ranges
7	3.0–3.2	Keep validation transformation independent of training fit
8	Export blocks	Update paths and legacy variable names before running

22. Quality-control checklist

- Confirm identical feature selection and feature order for training and validation.
- Confirm no missing or infinite values after import and normalization.
- Confirm every spectrum has non-zero norm.
- Confirm $\text{len}(\text{expanded Parameter A labels}) = \text{len}(\text{expanded Parameter B labels}) = \text{number of training spectra}$.
- Confirm spectra are ordered in contiguous condition blocks before Module 2.3.
- Fit StandardScaler only on training data.
- Use the same scaler and LDA objects for validation transformation.
- Check class counts and ensure every intended bin is represented.
- Inspect LD1/LD2 score plots, residuals and validation distributions rather than relying on R^2 alone.
- Resolve all legacy variable-name inconsistencies before a clean top-to-bottom execution.
- Record the exact preprocessing branch, bin edges, polynomial degrees and data split used for each reported result.

23. Limitations and recommended improvements

Area	Current implementation	Recommended improvement
Reproducibility	Hard-coded absolute paths and manually selected cells	Central configuration object and relative paths
Notebook structure	Executable code in Markdown cells; alternative branches overwrite variables	Convert all code to Code cells and expose preprocessing mode as a parameter
Naming	RPM/pressure/P and temperature/T names are mixed	Adopt Parameter A/Parameter B or one consistent domain vocabulary
Grouping	Condition statistics rely on fixed positional blocks	Use an explicit metadata table and groupby condition ID
Validation	Mixed assumptions of 1 versus 350 spectra per condition	Define validation layout once and derive grouping from metadata
Model selection	Polynomial degree chosen using in-sample diagnostics	Use grouped cross-validation or nested validation
Classification diagnostics	Imports exist but confusion matrix/cross-validation are absent	Add stratified or group-aware classification metrics
Leakage audit	Exact duplicates only	Add group-level split checks, near-duplicate detection and provenance IDs
Persistence	Models are not serialized	Save scaler, LDA estimators, regression objects and configuration together
Error handling	Minimal shape and range validation	Add assertions and informative exceptions

24. Output reference

Output family	Typical contents	Purpose
Class-balance CSV	Bin identifiers and counts	Document supervised class composition
Explained-variance CSV	LD component and variance ratio	Component-selection record
LD loading CSV	Feature index and coefficient	Spectral interpretation
LD1/LD2 projection CSV	Scores and class bins	External score-space plotting
Condition scatter CSV	Parameter A, Parameter B, LD1 mean and std	Calibration and 3D plotting
LD1-A versus LD1-B CSV	Paired latent scores, bins and optional colours	Correlation/decoupling analysis
Validation CSV	True, predicted and residual values	Independent performance reporting
Validation LD summary	Block sizes and LD1 means/stds	Condition-level validation comparison
Scaled-spectrum CSV	Processed feature matrix in transposed format	External visualization and archival

25. Dependency summary

Module 1.0 supplies raw spectra and labels. Module 1.1 supplies normalized sample-by-feature matrices. Module 1.2 supplies supervised class targets. Module 2.0 produces the scaler, LDA models and score projections. Modules 2.1 and 2.2 diagnose those models directly. Module 2.3 converts scores into condition-level statistics, which feed Modules 2.4, 2.5 and 3.0. Modules 2.6a and 2.6b use both condition-level and spectrum-level responses to assess decoupling. Module 3.0 applies all training-derived transforms to independent data. Modules 3.1 and 3.2 add distributional validation and leakage checks. Module 4.0 exports selected processed matrices.

26. Summary

The notebook implements a scientifically meaningful dual-LDA strategy for extracting two latent responses from a shared cross-sensitive spectral signal. Its major strengths are the use of separate supervised models, condition-level latent-score averaging, explicit confounding checks, independent validation, and extensive data export. Its principal weaknesses are operational rather than conceptual: inconsistent variable names, alternative executable sections stored as Markdown, hard-coded dataset geometry, and conflicting validation layouts. Once these are standardized, the workflow can serve as a reproducible analysis pipeline for two-parameter spectral sensing.